

in unharness mode by editing `android.buildTypes.release` in `[project]/android/app/build.gradle`.

- On iOS, produce and sign using a distribution certificate rather than a development certificate once you're able to take a look at and deploy mistreatment TestFlight or App Store.
 - Create and transfer a distribution certificate in your Apple Developer Account console.
 - Open `[project]/ios/Runner.xcworkspace/` and choose the distribution certificate in your target's settings pane.
7. Produce a Fastfile script for every platform.
- On Android, you'll need to follow the Fastlane android beta development guide. Though, your edit might be as straightforward as adding a lane that calls `upload_to_play_store` and set your apk argument to `../build/app/outputs/apk/release/app-release.apk` to use the apk flutter build already engineered.
 - On iOS, you'll need to consider the fastlane iOS beta deployment guide. Like Android, here your edit might be as straightforward as adding a lane that calls `build_ios_app` with `export_method: 'app-store'` and upload to testflight.

On iOS an additional build is needed because flutter build builds associate degree .app instead of archiving .ipas for the release. You're currently able to perform readings regionally or migrate the deployment method to a nonstop integration (CI) system.

Running Application Deployment Locally

1. Building the release mode app.
 - `flutter build apk --release`.
 - `flutter build ios --release --no-codesign`. No need to sign at that time itself since fastlane can sign once archiving.
2. Run the Fastfile script on every platform.
 - `cd android` then `fastlane [name of the lane you created]`.
 - `cd ios` then `fastlane [name of the lane you created]`.

Cloud build and deploy setup

Firstly, you need to follow the instructions in the local setup section described in 'Local setup' to make sure the method works before you can migrate on to a cloud system like Travis.